LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack and Queue

Oleh:

Rahma Syarita Jaya NIM. 2510817320004

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Rahma Syarita Jaya
NIM : 2510817320004

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1: Implementasi Stack 6
A. Source Code 6
B. Pembahasan 7
SOAL 2: Si Palui dan Mesin Hitung Aneh 9
A. Source Code 10
B. Output Program 11
C. Pembahasan 12
SOAL 3: Implementasi Queue 13
A. Source Code 13
B. Pembahasan 14
SOAL 4: Si Palui dan Deretan Angka 16
A. Source Code 17
B. Output Program 18
C. Pembahasan 18
TAUTAN GIT 20

DAFTAR GAMBAR

Gambar 1.1 Screenshot Hasil Jawaban Soal 1A 11
Gambar 2.1 Screenshot Hasil Jawaban Soal 2A 18

DAFTAR TABEL

Tabel 1 Source code stack.cpp 6
Tabel 2 Source code prob1.cpp 10
Tabel 3 Source code queue.cpp 13
Tabel 4 Source code prob2.cpp 17

SOAL 1: Implementasi Stack

Soal 1: Implementasi Stack Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s){
    s->top = nullptr;
}

bool isEmpty(const Stack* s){
    return s->top == nullptr;
}

bool isFull(const Stack* s){
    if (isEmpty(s)) {
        return false;
    }
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::overflow_error("Penuh");
    }

    if (isEmpty(s)){
        s->top = &(s->data[0]);
    } else {
        s->top++;
    }
    *(s->top) = value;
}

void pop(Stack* s){
    if (isEmpty(s)) {
        throw std::underflow_error("Kosong");
```

```
35      }
36      if (s->top == &(s->data[0])) {
37          s->top = nullptr;
38      } else {
39          s->top--;
40      }
41  }
42
43  int peek(const Stack* s){
44      if (isEmpty(s)) {
45          throw std::underflow_error("Kosong");
46      }
47      return *(s->top);
48  }
```

B. Pembahasan

Pada baris [1 sampai 2] `stack.cpp`, mengimpor file header `"stack.h"` untuk mengenali struct `Stack` dan mengimpor library `<stdexcept>` untuk memunculkan pesan error dengan throw.

Pada baris [4 sampai 6] `stack.cpp`, fungsi `init(Stack* s)` menginisialisasi tumpukan kosong dengan mengatur pointer top menjadi `nullptr`.

Pada baris [8 sampai 10] `stack.cpp`, terdapat fungsi `isEmpty(const Stack* s)` bertipe boolean dimana akan mengembalikan nilai `true` jika pointer top menunjuk ke `nullptr` atau pointer kosong yang tidak menunjuk ke alamat memori manapun.

Pada baris [12 sampai 17] `stack.cpp`, fungsi `isFull(const Stack* s)` mengecek apakah kapasitas array sudah penuh dengan membandingkan alamat pointer top terhadap alamat elemen terakhir `array &(s->data[MAX - 1])`. Mengapa harus `[MAX - 1]` dan tidak langsung `MAX` saja? Ini karena komputer mulai menghitung dari angka 0, bukan angka 1 atau konsep ini disebut Zero-Based Indexing.

Pada baris [19 sampai 31] `stack.cpp`, fungsi `push(Stack* s, int value)` berfungsi untuk memasukkan data baru. Dimana pertama-tama mengecek apakah batas memori masih cukup. Jika iya, pointer top akan digeser naik dan nilai baru dimasukkan.

Pada baris [33 sampai 44] `stack.cpp`, fungsi `pop(Stack* s)` berfungsi untuk menghapus data paling atas. Pertama-tama fungsi ini akan melakukan `throw std::underflow_error("Kosong")` jika tumpukan sudah kosong. Jika ada isinya, fungsi ini akan menghapus data dengan menggeser pointer `top` turun satu Tingkat `s->top--`.

Pada baris [46 sampai 52] `stack.cpp`, terdapat fungsi `peek(const Stack* s)` yang berfungsi untuk mengembalikan nilai data di posisi paling atas tumpukan tanpa mengubah posisi pointer dan menghapusnya dengan mengakses nilai yang ditunjuk oleh operator pointer `*(s->top)`, angka di atas tumpukan tetap ada dan tidak terhapus.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)
- Cara kerja mesin adalah sebagai berikut:
- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, alu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir. Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
- bilangan bulat
- operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

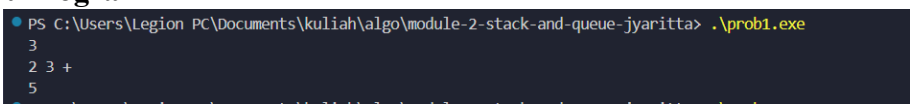
A. Source Code

Tabel 2 Source code prob1.cpp

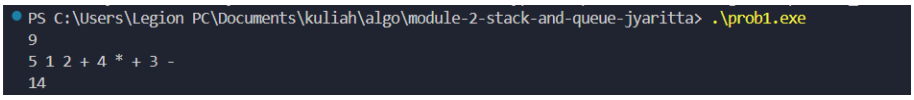
```
#include <iostream>
#include <string>
#include "stack.h"

using namespace std;

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Stack memory;
    init(&memory);

    for (int i = 0; i < n; i++) {
        string token;
        cin >> token;

        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int A2 = peek(&memory);
            pop(&memory);

            int A1 = peek(&memory);
            pop(&memory);
```

```

            int hasil = 0;

            if (token == "+") {
                hasil = A1 + A2;
            } else if (token == "-") {
                hasil = A1 - A2;
            } else if (token == "*") {
                hasil = A1 * A2;
            } else if (token == "/") {
                hasil = A1 / A2;
            }

            push(&memory, hasil);

        } else {
            push(&memory, stoi(token));
        }
    }

    cout << peek(&memory) << endl;

    return 0;
}
```

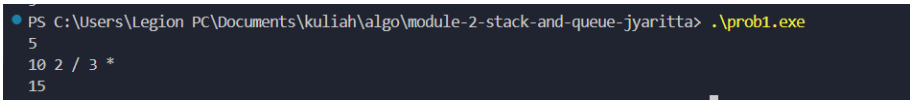
B. Output Program



Gambar 1.1 Screenshot Hasil Jawaban Soal 1A



Gambar 1.2 Screenshot Hasil Jawaban Soal 1B



Gambar1.3 Screenshot Hasil Jawaban Soal 1C

C. Pembahasan

Pada baris [1 sampai 3] `prob1.cpp`, syntax `#include <iostream>` untuk mengimpor input/output, `<string>` untuk bisa manipulasi string/teks, dan `"stack.h"` agar program utama mengenali fungsi yang telah dibuat.

Pada baris [5] `prob1.cpp`, syntax `using namespace std;` digunakan agar tidak menuliskan ulang `std::` lagi pada perintah `C++.`

Pada baris [7 sampai 54] `prob1.cpp`, terdapat fungsi utama `int main()` yang merupakan titik awal program akan pertama kali dieksekusi oleh komputer.

Pada baris [8 sampai 10] `prob1.cpp`, program mendeklarasikan variabel `n` bertipe `integer` untuk membaca jumlah ukuran yang diinputkan user pada baris pertama.

Pada baris [12 sampai 13] `prob1.cpp`, program membuat variabel memory bertipe `Stack` dan langsung memasukkan alamat memorinya ke dalam fungsi `init(&memory)` agar tumpukannya disiapkan dalam keadaan kosong.

Pada baris [16 sampai 48] `prob1.cpp`, terdapat perulangan `for` yang berjalan sebanyak `n` kali untuk membaca input berupa `string` token. Jika `token` berupa operator `(+, -, *, /),` program akan memanggil fungsi `pop` dua kali untuk mengambil angka `A1dan A2` dari tumpukan memori.

Pada baris [33 sampai 43] `prob1.cpp`, terdapat implementasi logika percabangan aritmatika dimana program mengecek isi variabel `token`. Jika token berisi simbol penjumlahan, pengurangan, perkalian, atau pembagian, maka program akan memproses variabel `A1` dan `A2` sesuai operasi aritmatika. Khusus untuk pembagian, hasil akan dibulatkan sesuai sifat integer pada C++. Hasil operasi tadi didorong kembali ke dalam stack memori dan memastikan nilai tersebut siap dieksekusi jika terdapat operator lain di belakangnya.

Pada baris [45 sampai 48] `prob1.cpp`, pada syntax else, jika simbol `token` yang dibaca adalah angka maka program akan mengubah teks tersebut menjadi angka `integer` menggunakan fungsi `stoi()` dan memasukkannya ke tumpukan.

Pada baris [51] `prob1.cpp`, syntax `cout` berguna untuk mencetak satu angka terakhir yang tersisa di dalam memori dengan fungsi `peek(&memory).`

Pada baris [53] `prob1.cpp`, terdapat syntax `return 0;` untuk mengakhiri fungsi main bahwa program telah berjalan dan selesai tanpa adanya error.

SOAL 3: Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3 Source code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;
    return q->rear == &(q->data[MAX - 1]);
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw std::overflow_error("Penuh");
    }

    if (isEmpty(q)) {
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
    } else {
        q->rear++;
    }
    *(q->rear) = value;
}
void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Kosong");
```

```
34      }
35
36      if (q->front == q->rear) {
37          q->front = nullptr;
38          q->rear = nullptr;
39      } else {
40          q->front++;
41      }
42  }
43
44  int front(const Queue* q) {
45      if (isEmpty(q)) {
46          throw std::underflow_error("Kosong");
47      }
48      return *(q->front);
49  }
50
51  int back(const Queue* q) {
52      if (isEmpty(q)) {
53          throw std::underflow_error("Kosong");
54      }
55      return *(q->rear);
56  }
```

B. Pembahasan

Pada baris [1 sampai 2] `queue.cpp`, mengimpor file header `"queue.h"` untuk mengenali struct `queue` dan mengimpor library `<stdexcept>` untuk memunculkan pesan error dengan `throw`.

Pada baris [4 sampai 7] `queue.cpp,` fungsi `init(Queue* q)` berfungsi untuk menginisialkan antrean menjadi kosong dengan cara membuat pointer front (depan) dan rear (belakang) menunjuk `nullptr`.

Pada baris [9 sampai 11] `queue.cpp`, fungsi `isEmpty(const Queue* q)` bergunsi untuk memvalidasi apakah antrean kosong dengan cara melihat pointer front apakah itu bernilai `nullptr`. Jika iya, fungsi mengembalikan nilai `true`.

Pada baris [13 sampai 16] `queue.cpp`, fungsi `isFull(const Queue* q)` digunakan untuk memvalidasi batas memori array. Antrean akan dinyatakan penuh atau true jika pointer rear telah berada pada bagian paling akhir array `&(q->data[MAX - 1])`.

Pada baris [18 sampai 30] `queue.cpp`, fungsi `enqueue(Queue* q, int value)` berfungsi untuk memasukkan elemen baru dari bagian belakang antrean. Fungsi diawali dengan memanggil `isFull`, jika batas tercapai, program akan memicu `throw std::overflow_error()`. Jika masih ada memori array, pointer `front` mengarah ke

indeks `0` atau deretan paling pertama dan pointer `rear` akan mengarah ke indeks `0` atau deretan paling akhir. Jika ada data masuk, pointer `rear` akan bergeser naik `(rear++)`.

Pada baris [31 sampai 41] `queue.cpp`, `dequeue(Queue* q)` digunakan untuk mengeluarkan elemen paling depan. Di dalam fungsi ini terdapat `isEmpty` dimana akan melakukan `throw std::underflow_error()` jika antrean kosong. Jika elemen sudah tersisa satu `front == rear`, pointer dikembalikan ke `nullptr`. Jika elemen lebih dari satu, pointer front digeser naik untuk mengabaikan data terdepan yang lama.

Pada baris [43 sampai 55] `queue.cpp`, terdapat fungsi `front()` dan `back()` yang masing-masing berfungsi untuk melihat nilai dengan menggunakan operator dereference atau tanda `*` dari elemen paling depan dan paling belakang pada antrean tanpa mengubah struktur pointer, dilengkapi dengan penanganan error jika antrean kosong.

SOAL 4: Si Palui dan Deretan Angka

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

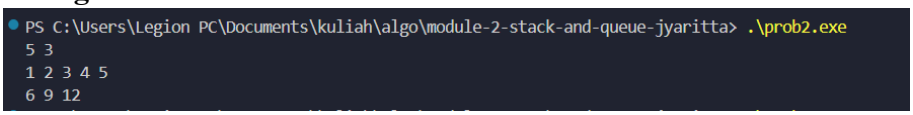
A. Source Code

Tabel 4 Source code prob2.cpp

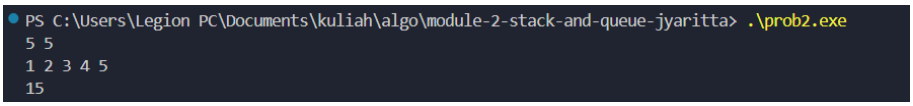
```
#include <iostream>
#include "queue.h"

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) return 0;
    Queue window;
    init(&window);
    long long current_sum = 0;
    int temp;

    for (int i = 0; i < k; i++) {
        cin >> temp;
        enqueue(&window, temp);
        current_sum += temp;
    }
    cout << current_sum;

    for (int i = k; i < n; i++) {
        cin >> temp;
        current_sum -= front(&window);
    ~    dequeue(&window);
        enqueue(&window, temp);
        current_sum += temp;
        cout << " " << current_sum;
    }
    cout << endl;
```

```
30      return 0;
31  }
```

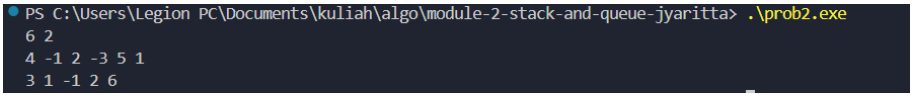
B. Output Program



Gambar 2.1 Screenshot Hasil Jawaban Soal 2A



Gambar2.2 Screenshot Hasil Jawaban Soal 2B



Gambar 2.3 Screenshot Hasil Jawaban Soal 2C

C. Pembahasan

Pada baris [1 sampai 4] `prob2.cpp`, syntax `#include <iostream>` untuk mengimpor input/output dan memanggil header `"queue.h"` agar program utama mengenali fungsi yang telah dibuat.

Pada baris [6 sampai 8] `prob2.cpp`, terdapat deklarasi fungsi utama `int main()` serta inisialisasi variabel `n` untuk jumlah total elemen array dan `k` untuk ukuran jendela geser, dan pembacaan input dari pengguna untuk elemen `n` dan `k`.

Pada baris [9 sampai 12] `prob2.cpp`, program mendeklarasikan variabel `window` dari tipe data `Queue` dan melakukan inisialisasi menggunakan `init(&window)`. mendeklarasikan pula variabel `current_sum` bertipe `long long` untuk kapasitas perhitungan jika nantinya ada akumulasi angka bernilai besar.

Pada baris [14 sampai 18] `prob2.cpp`, program mengeksekusi langkah pertama dari algoritma menggunakan perulangan `for`. Program memakai `enqueue` sebanyak `k` angka pertama ke dalam antrean, lalu langsung menjumlahkannya ke dalam variabel `current_sum`.

Pada baris [19] `prob2.cpp`, program mencetak `current_sum` yang merupakan total dari jendela pertama ke layar terminal.

Pada baris [21 sampai 28] `prob2.cpp`, program masuk ke langkah selanjutnya yaitu setiap iterasi mulai dari indeks ke-`k` hingga `n`, program akan mengurangi nilai paling depan dari total saat ini atau `current_sum -= front(&window)`, lalu mengeluarkan elemen tersebut dari memori dengan `dequeue`.\

Pada baris [25 sampai 27] `prob2.cpp`, secara bersamaan program membaca satu angka baru yang masuk ke dalam `window`, menyimpannya ke antrean bagian belakang atau `enqueue`, lalu menambahkannya ke perhitungan `current_sum`, kemudian langsung mencetak hasil perhitungan yang baru yang akan dipisah dengan spasi agar angka hasil perhitungan sejajar ke samping dalam satu baris.

Pada baris [29 sampai 31] `prob2.cpp`, program memberikan baris baru di akhir keluaran dengan `endl` sekaligus membersihkan buffer keluaran dan menutup fungsi utama dengan `return 0`.

Penggambaran langkah-langkah window pada percobaan soal 2A adalah sebagai berikut.

1	$1 + 2 + 3 = 6$
2	$6 - 1 + 4 = 9$
3	$9 - 2 + 5 = 12$

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-jyaritta